



Stochastic Modeling of Glacier Melting

Course: CSE400 – Fundamentals of Probability in Computing
Domain: Climate

GROUP-12

Dhairya Shah, Dhanvi Joshi, Jahnavi Patel, Jashvi Shah, Manya Morakhia





Background and Motivation

- **Course Perspective**
 - Deterministic vs Randomized Algorithms
 - Uncertainty Modeling
 - Monte Carlo vs LHS
- **Real-World Perspective**
 - Glacier melt = uncertain climate inputs
 - Impacts:
 - Climate change
 - Water resources
 - Sea-level rise



Base Paper Overview

- Based on glacier mass balance modeling
- Uses energy balance approach
- Considers:
 - Radiation
 - Temperature
 - Precipitation
- **Key Contribution**
 - Physically grounded model for glacier melt
 - Shows importance of uncertainty & variability



Problem Formulation

The problem we address is predicting glacier melt under uncertain and variable climate conditions. Glacier melting depends on factors such as air temperature, precipitation, ice albedo, cloudiness, and snowfall threshold, all of which exhibit natural variability. Deterministic models use fixed inputs and produce a single estimate, which fails to capture uncertainty and extreme scenarios. Hence, a probabilistic approach is required to model variability and better understand the range and risk of glacier melt outcomes.



Assumptions

- Climate variables follow appropriate probability distributions (Normal, Gamma, Truncated Normal) and are assumed to be independent
- Glacier melt model is deterministic, with uncertainty introduced through inputs (including systematic and random errors)
- The model assumes non-linear sensitivity and some temporal correlation in variables

Objective

The objective is to develop a stochastic simulation framework to estimate glacier melt under uncertainty by generating probability distributions instead of a single value, identifying extreme scenarios, analyzing key variables, and improving accuracy and efficiency using Latin Hypercube Sampling over Monte Carlo methods.

Mathematical Model

SNOWFALL THRESHOLD

$$T_s \sim N(1.0, 0.5^2)$$

$$P_{snow}(T) = \Phi \left(\frac{T_s - T}{\sigma_t} \right)$$

Using Empirical Limit
Bounds: -0.5 to 2.5

AIR TEMPERATURE

$$T_a \sim N(\mu, \sigma^2)$$

Using Chebyshev's
inequality,

$$P(|T_a - \mu| \geq k\sigma) \leq 1/k$$

Bounds: $T_a \in [-1.3, 1.3]$

CLOUDINESS

$$n \sim N(0.5, 0.2^2)$$

$$T_{cl} = 1.0 - 0.233n - 0.415n^2$$

Using Logic Limit
Bounds: 0 to 1

Mathematical Model

Precipitation

Using Markov
Bounds: ≥ 0

Ice Albedo

Using Physical Limit
Bounds: 0.2 to 0.5

Mathematical Model

Melt Energy:

$$Q_{net} = S_0 \cdot \tau \cdot (1 - \alpha) + C_{sensible} \cdot T_a$$

Melt Mass:

$$M = \left(\frac{Q_m}{\rho_w L_f} \right) \Delta t$$

Seasonal Melt Magnitude: This is adjusted based on whether precipitation falls as rain or snow.



BOUNDS

CLOUDINESS : $0 \leq n \leq 1$

ICE ALBEDO : $\alpha \in [0.2, 0.5]$

PRECIPITATION : $P \geq 0$

SNOWFALL THRESHOLD: $[-0.5, 2.5]$

AIR TEMPERATURE: $T_a \in [-1.3, 1.3]$



Deterministic Approach

- Fixed input variables (T_a , P , α , n , T_s)
- Single output value

Limitations:

- No uncertainty
- No extreme events
- Not realistic



Randomized Approach (Monte Carlo)

- Inputs sampled from probability distributions
- Multiple simulations ($N \approx 5000$)
- Output = distribution of melt

Gives:

- Mean melt
- 95% risk

Limitations:

- Random clustering
- Misses extremes
- High variance



Randomized Approach (Latin Hypercube Sampling)

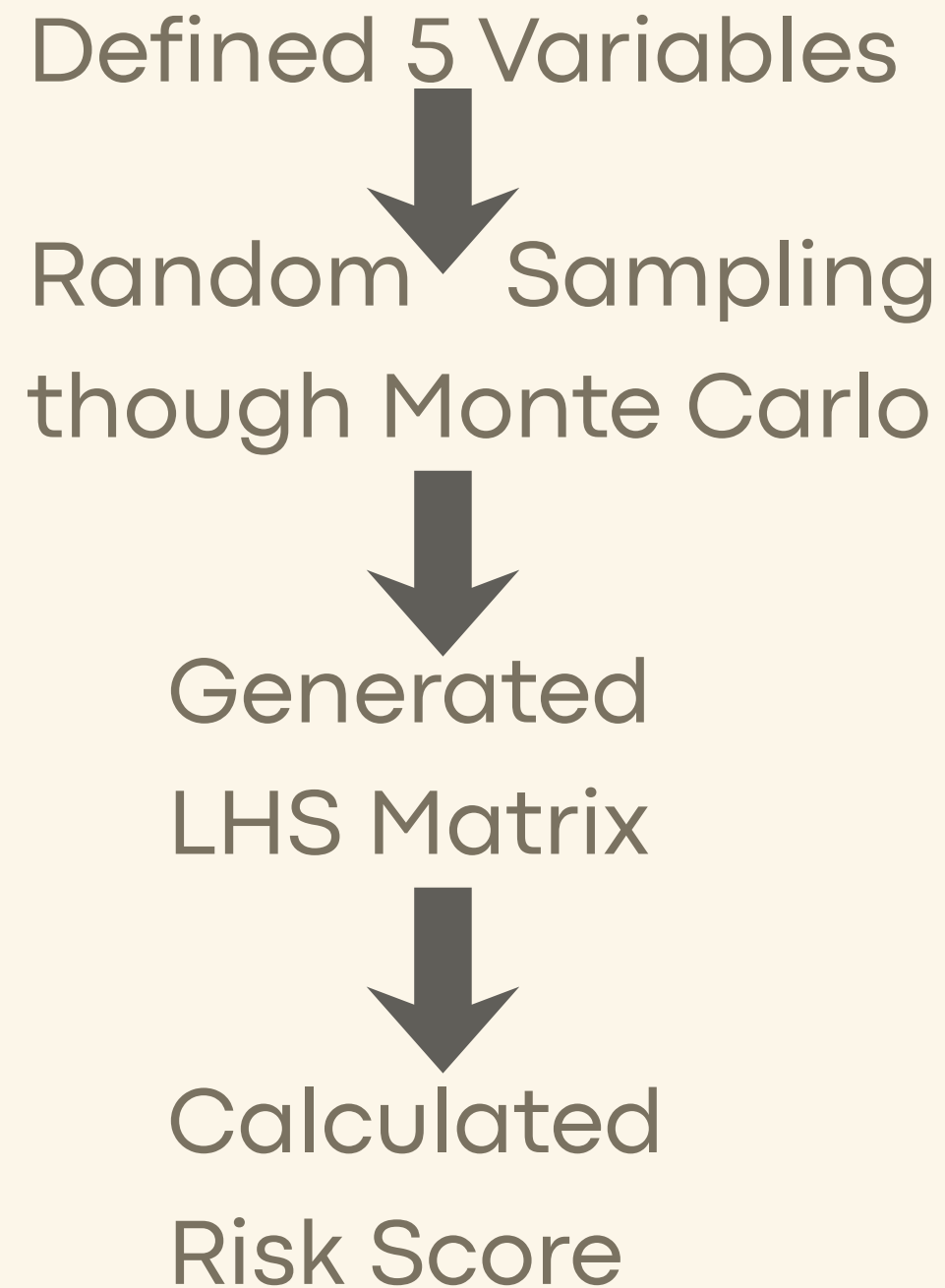
- Structured sampling
- Each interval sampled once
- $N \times 5$ input matrix

Advantages:

- Uniform coverage
- Captures extremes
- Lower variance
- Faster convergence

Coding and Simulation

Below is the pipeline that was followed to make the coding and simulation.





Coding and Simulation

Initially we used Independent Random Sampling to establish our baseline:

Monte Carlo Code Snippet

```
✓ import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm

✓ def simulate_thermal_dynamics_mc(mu_obs, sigma_est, k_coeff, n_trials=5000):
    samples = np.random.normal(loc=mu_obs, scale=sigma_est, size=n_trials)
```

Generate random samples for normal distribution

```
samples = np.random.gamma(k, theta, N)
```

Generate random samples for gamma distribution



Coding and Simulation

To improve stability, we implemented Stratified Latin Hypercube Sampling to coordinate the variables:

```
def run_lhs_glacier_model(n_samples: int = 5000)
    sampler = qmc.LatinHypercube(d=5)
    sample_matrix = sampler.random(n=n_samples)
```

Used Quasi Monte Carlo from
Scipy library

5 dimensions for 5 variables

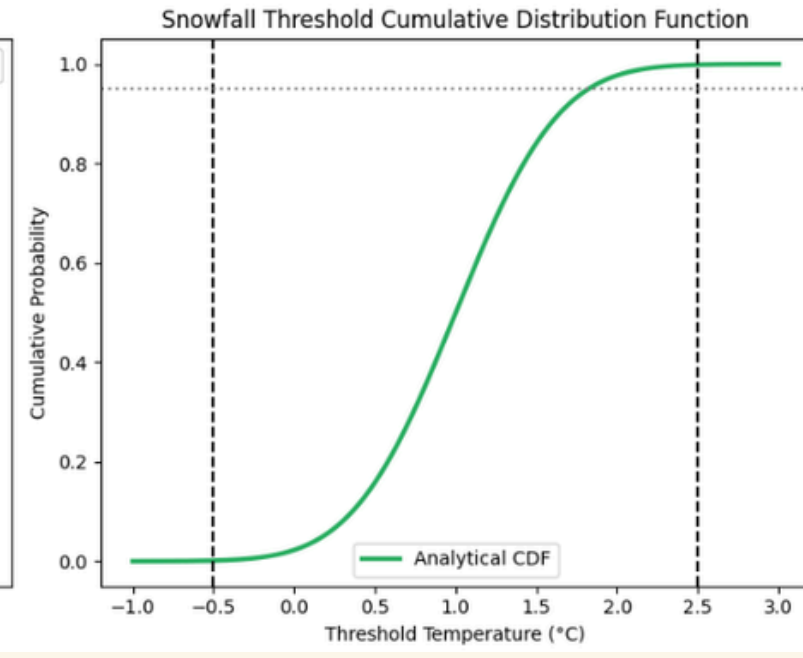
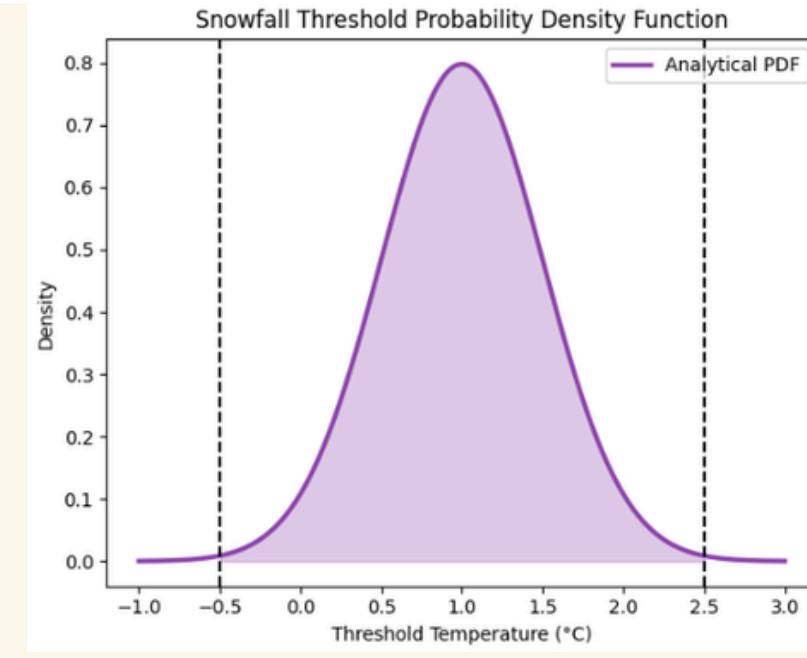
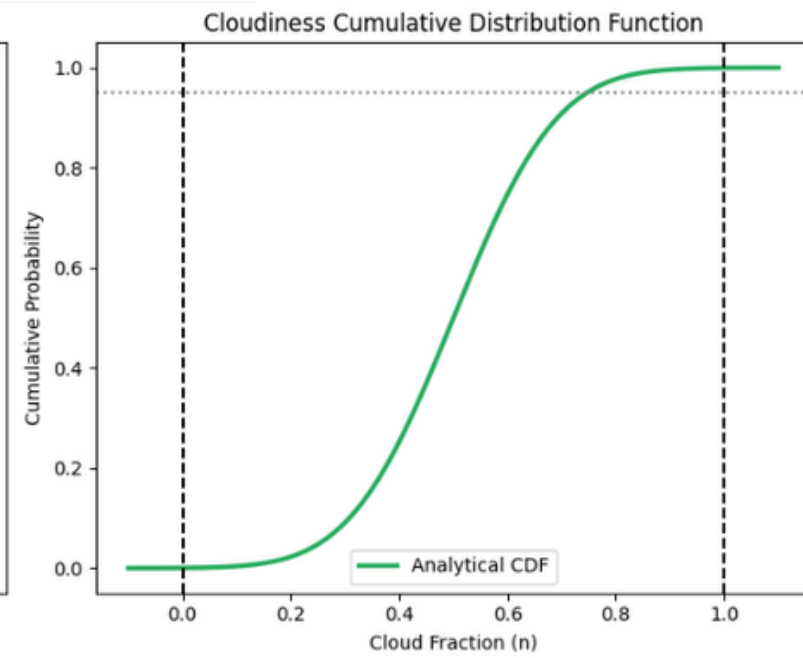
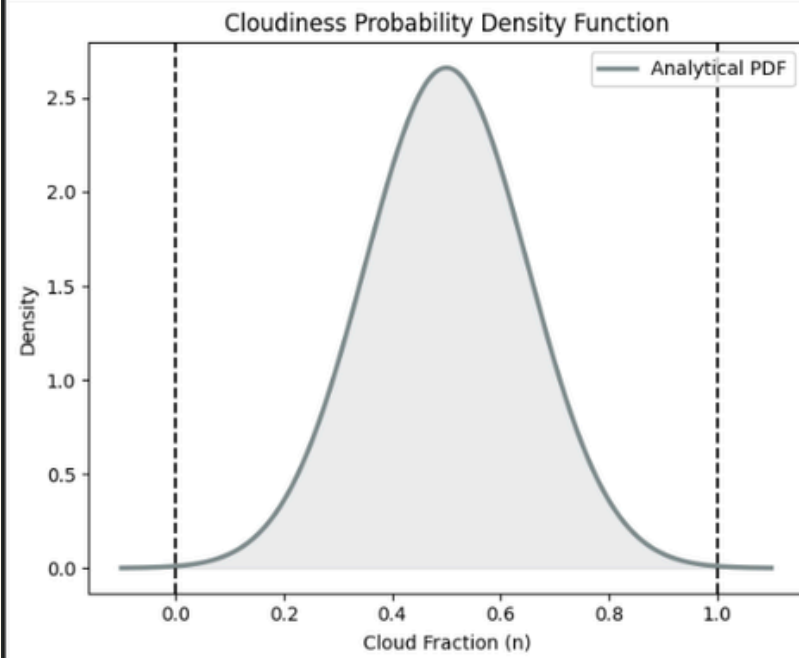
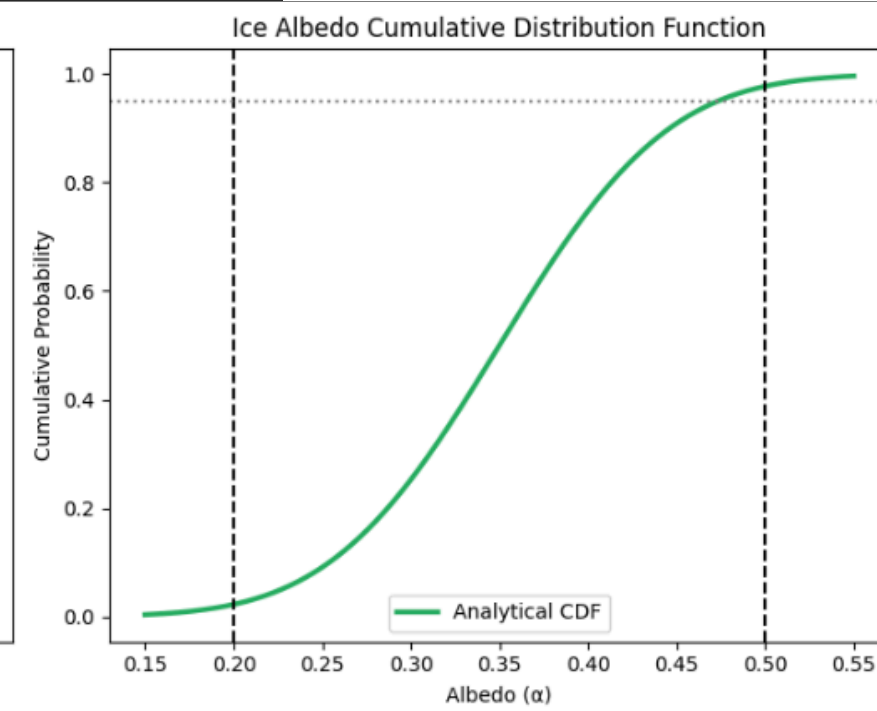
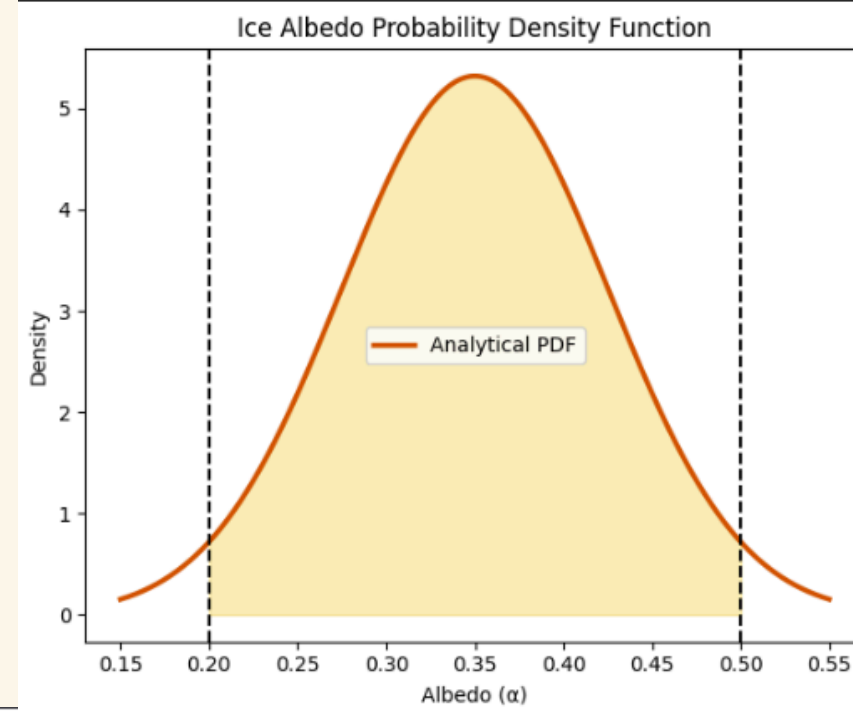
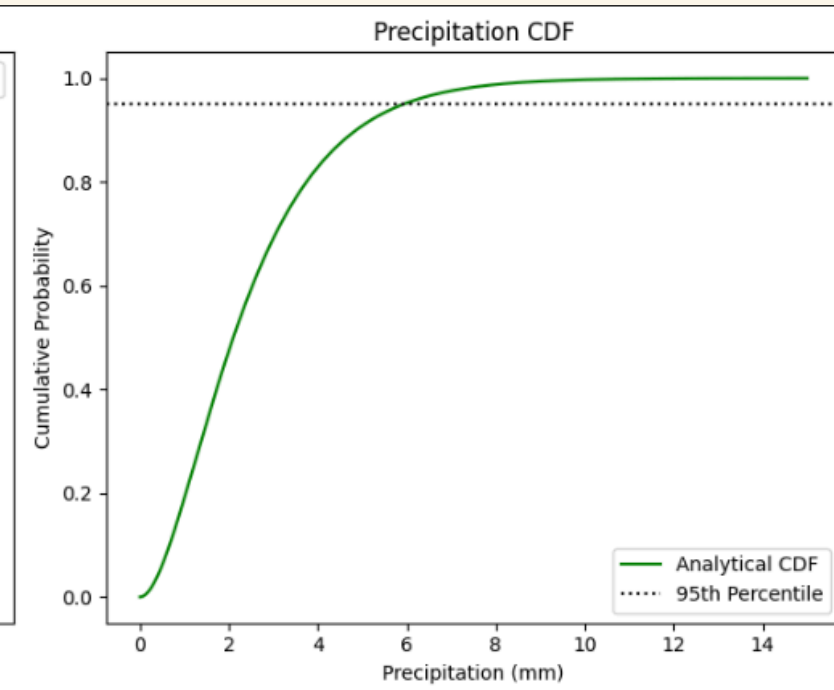
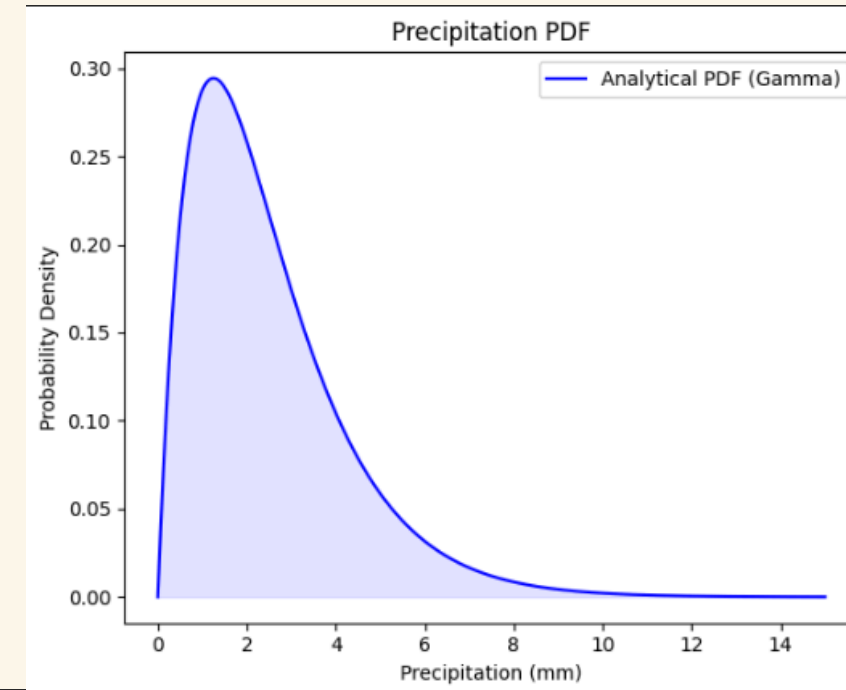
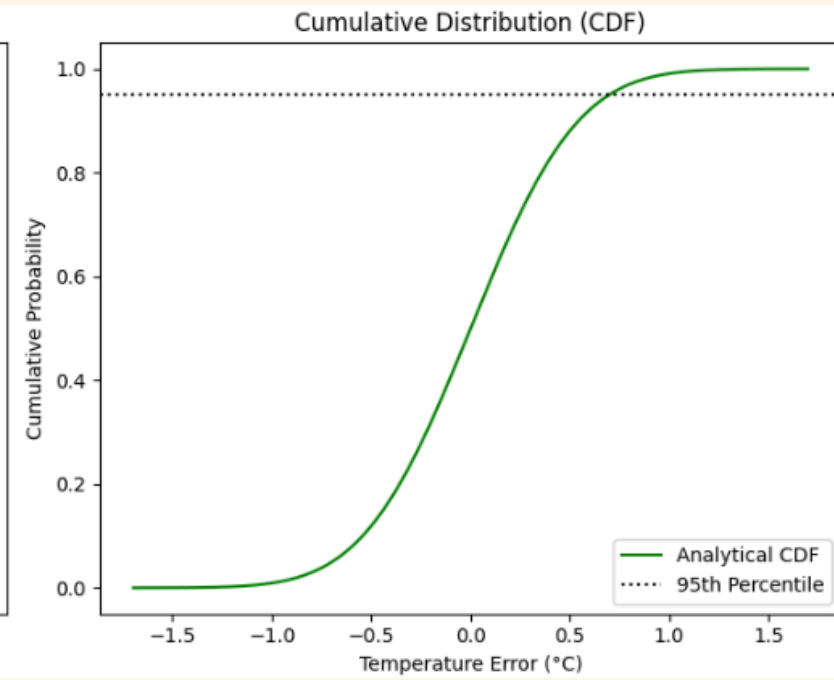
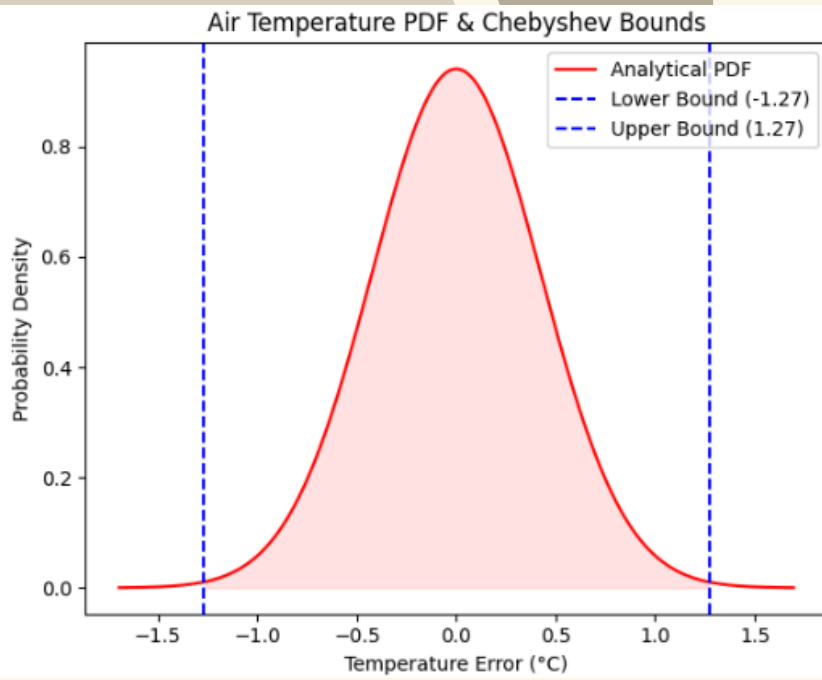


Coding and Simulation

This is how the code turns raw numbers into a physical glacier simulation:

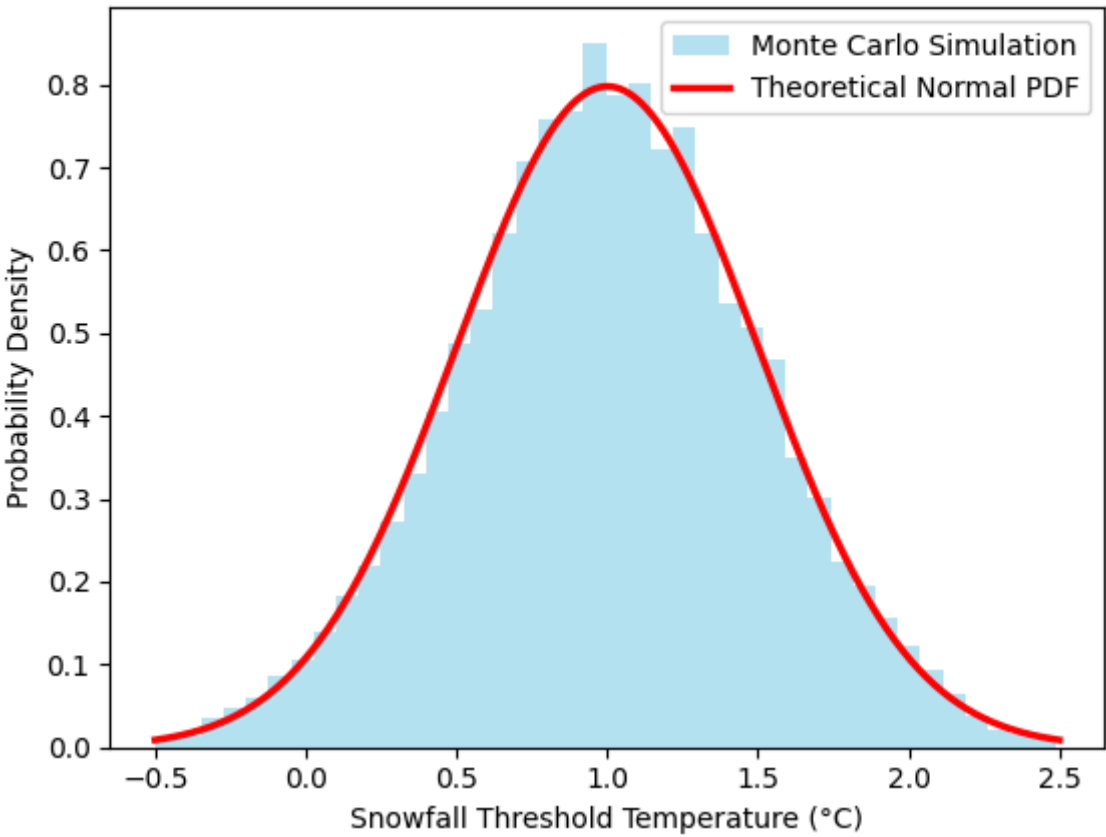
- `.ppf()` (Percent Point Function) translate the $[0,1]$ LHS matrix into physical units
- `np.clip` enforce Chebyshev and Physical limits, preventing impossible weather inputs.
- A conditional branch simulates the glacier's physical response deciding whether to add precipitation mass as rain or snow

Results

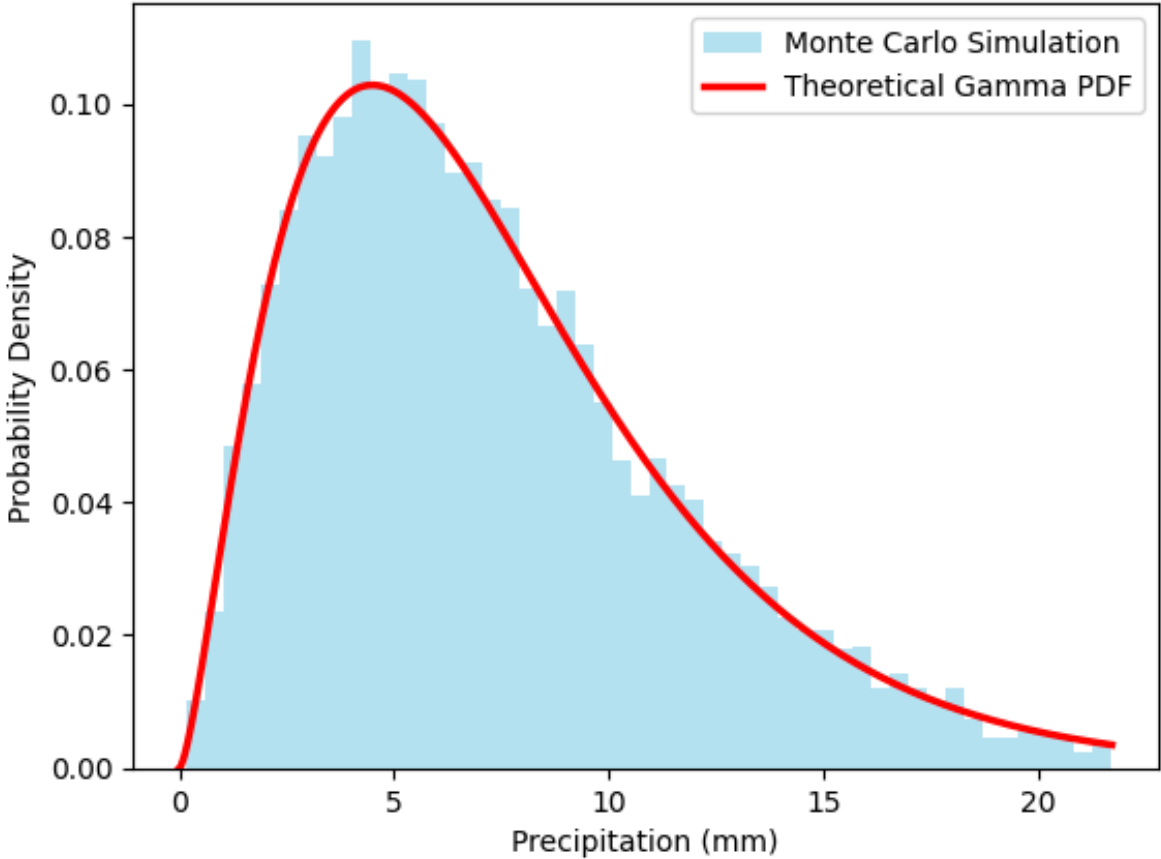


Results

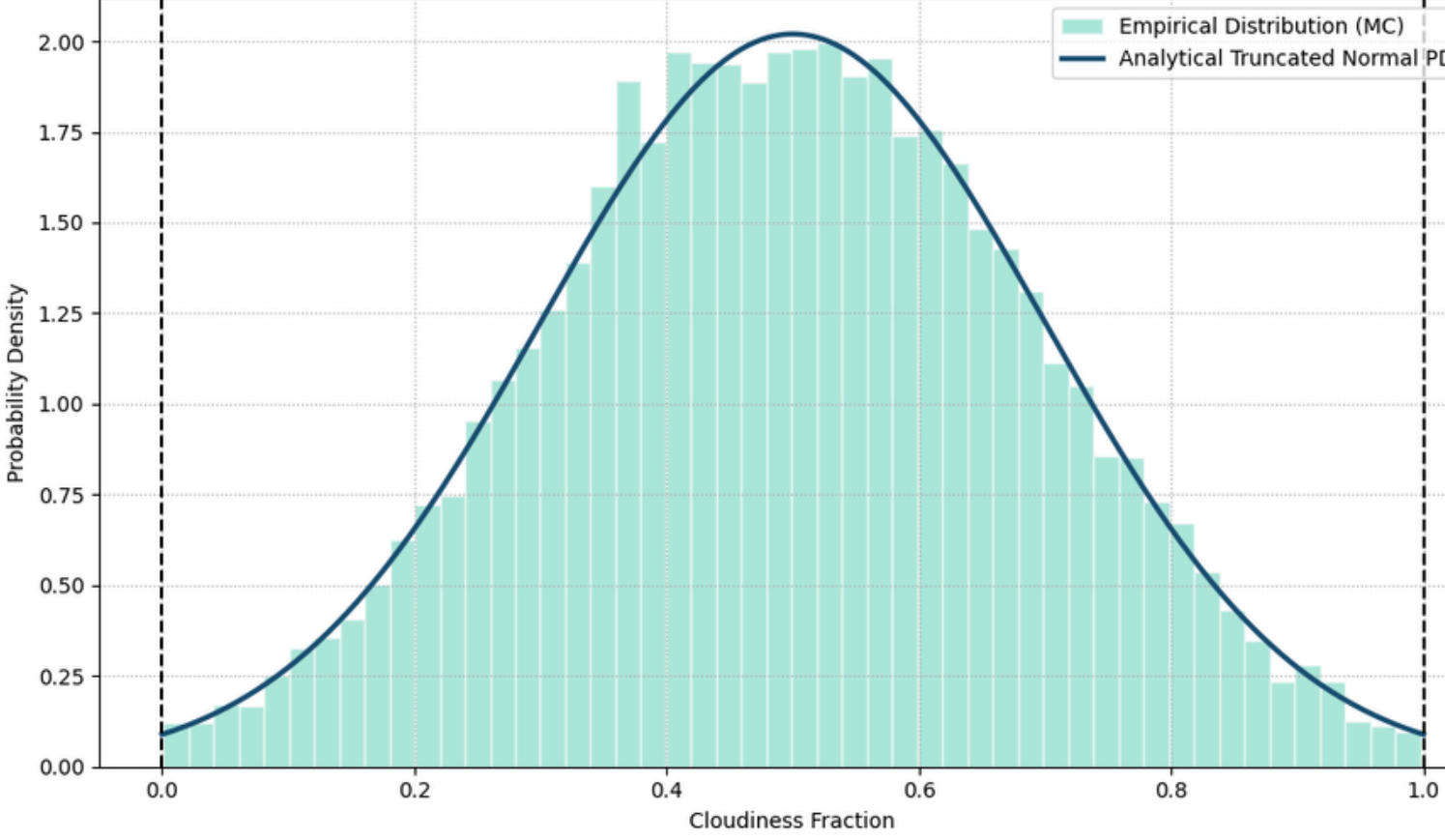
Monte Carlo Simulation vs Theoretical Distribution
Snowfall Threshold (T_s)



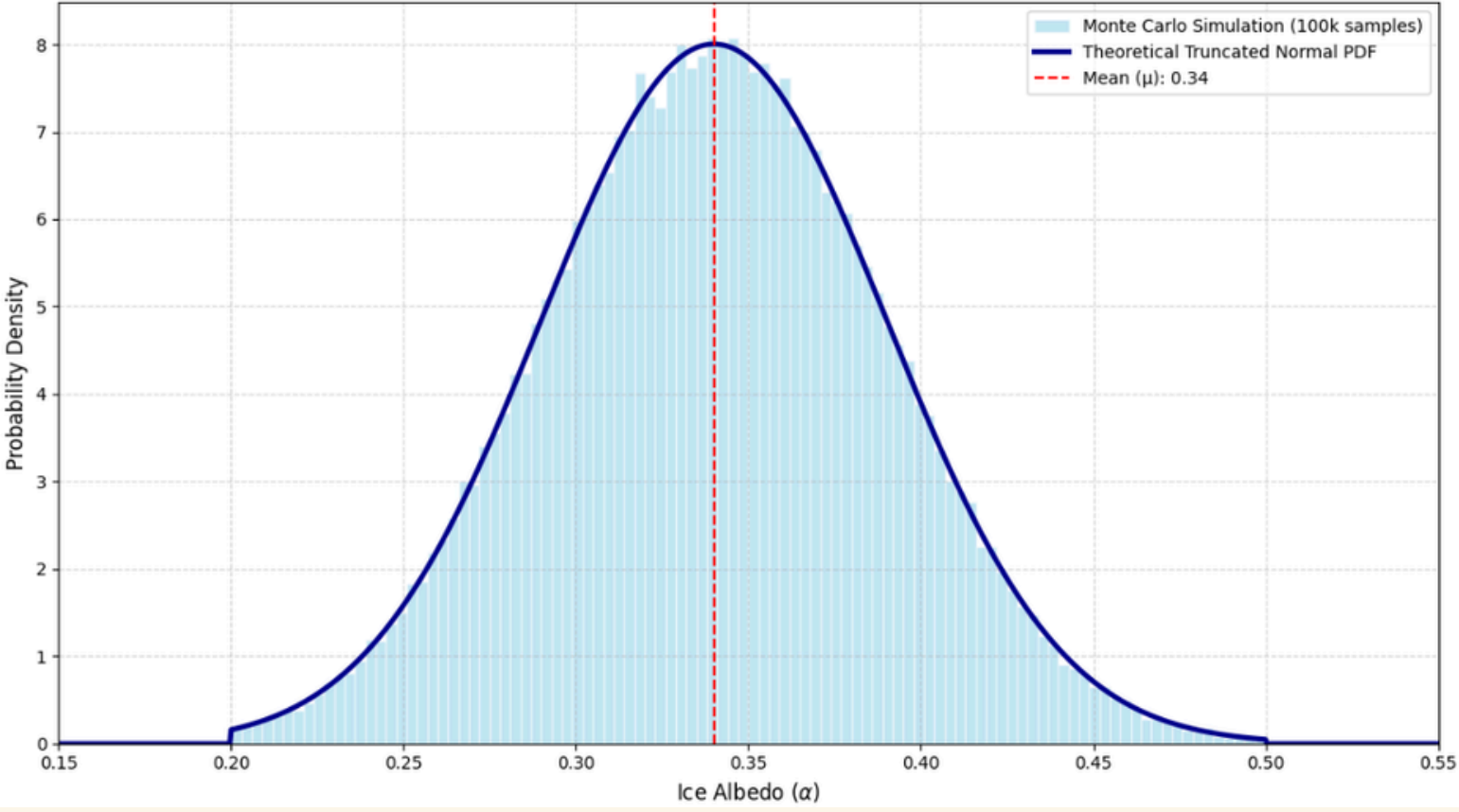
Monte Carlo Validation: Precipitation Distribution



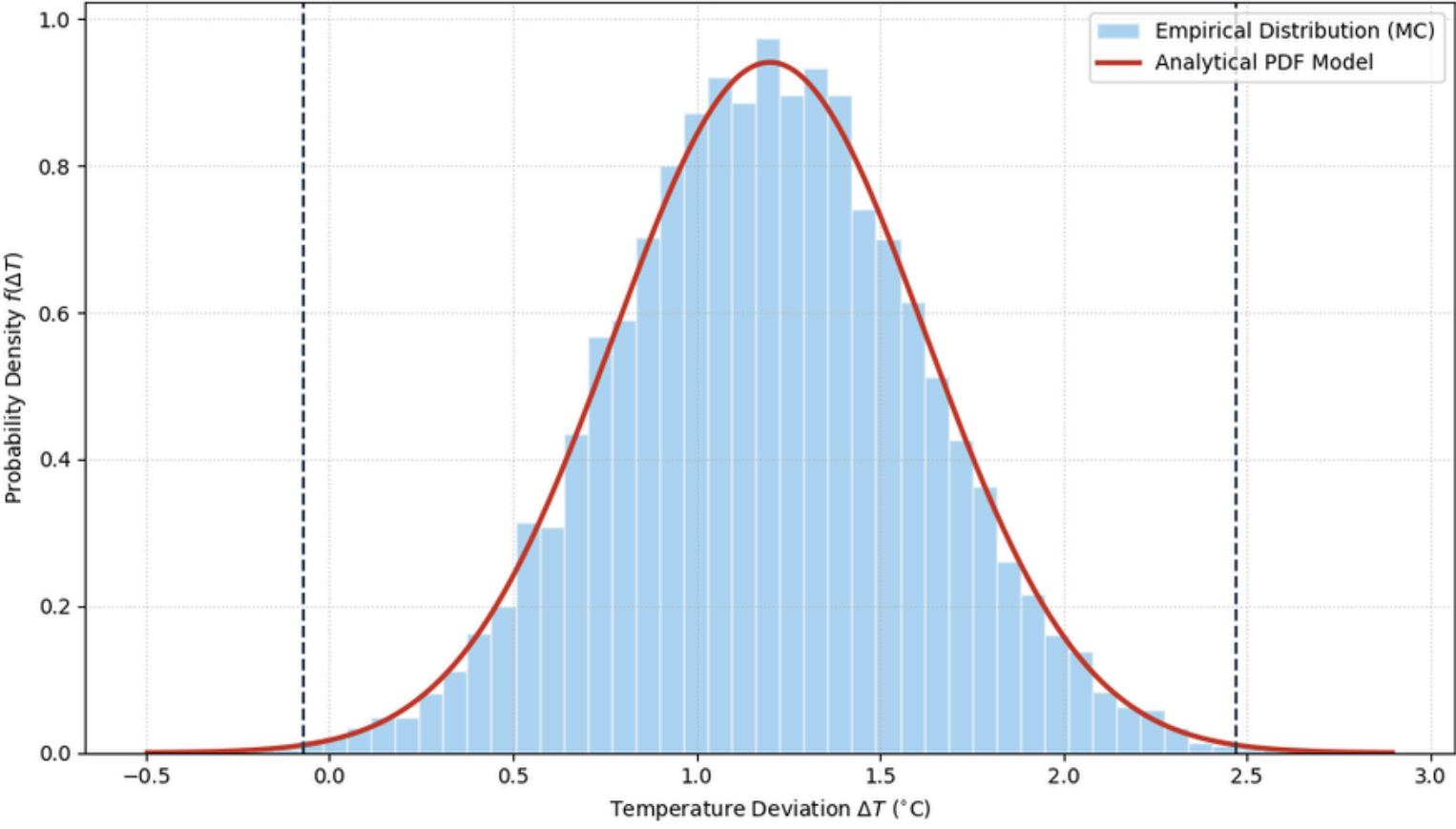
Monte Carlo Verification for Cloudiness (Bounded [0,1])



Validation of Ice Albedo Model: Monte Carlo vs. Theoretical PDF



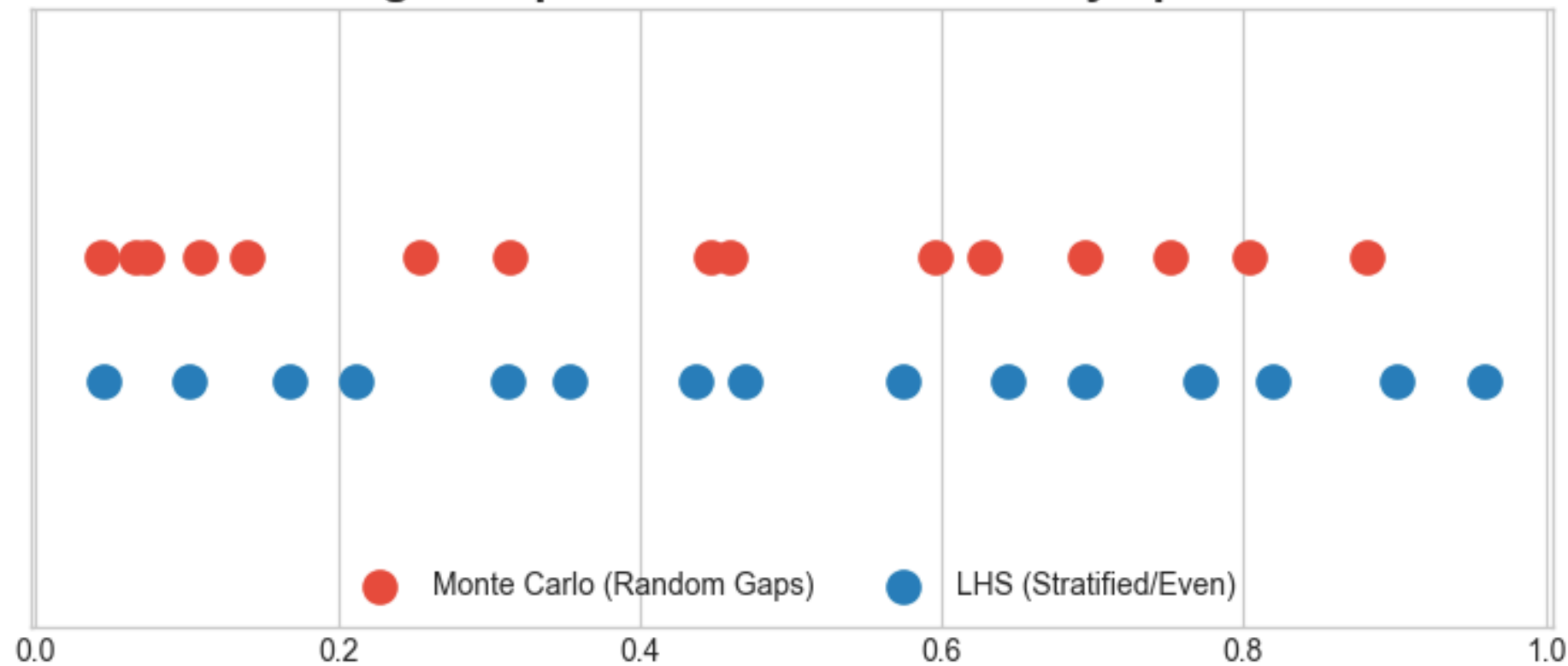
Stochastic Verification: Empirical vs. Analytical Thermal States



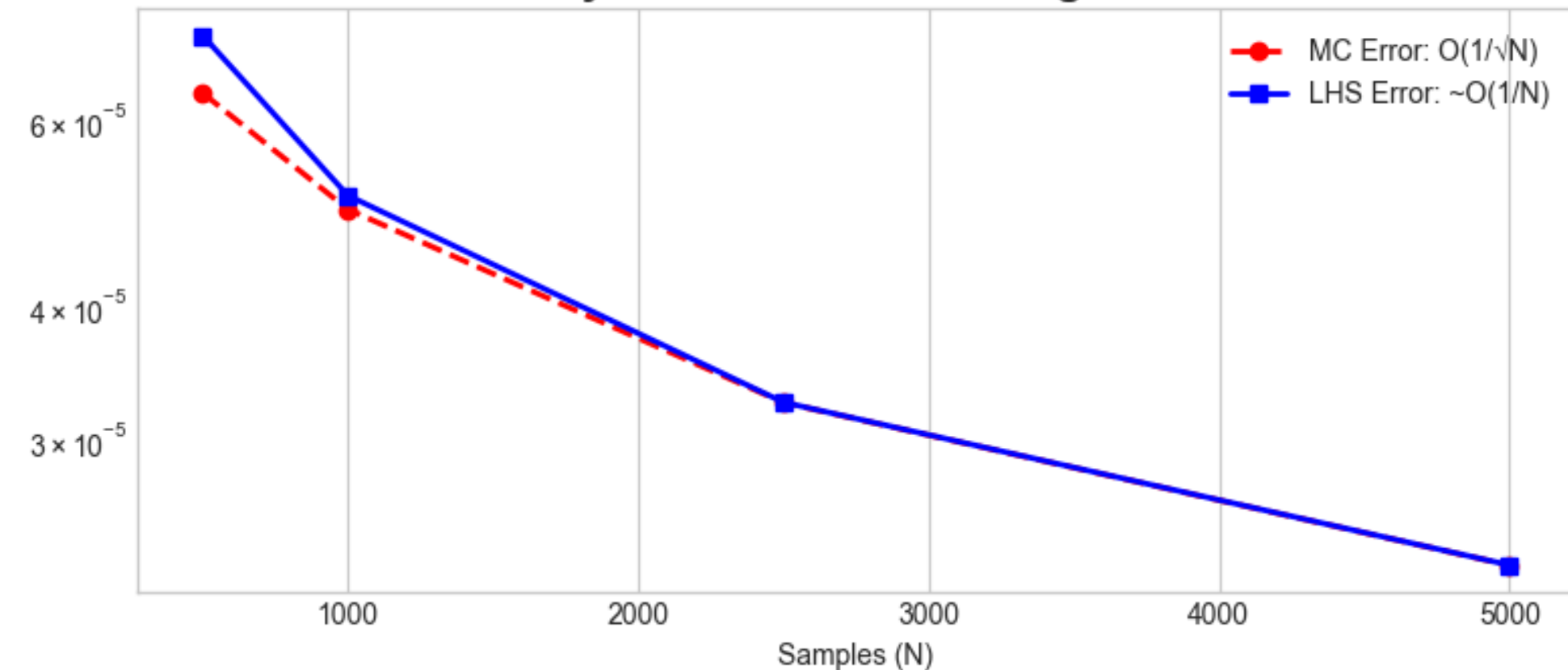
Results

Milestone 1-4: Strategic Simulation & Extreme Risk Analysis

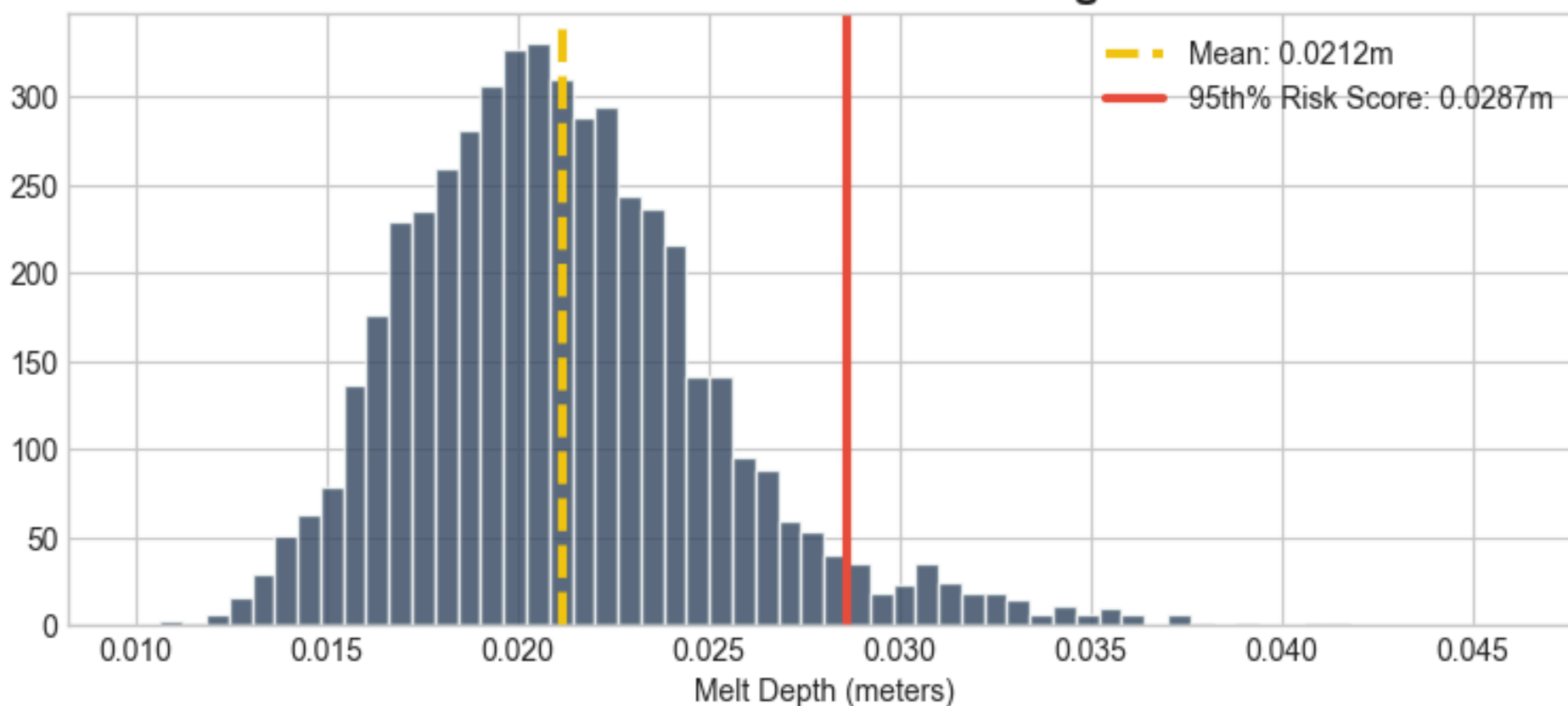
Logic: Gap Elimination in Probability Space



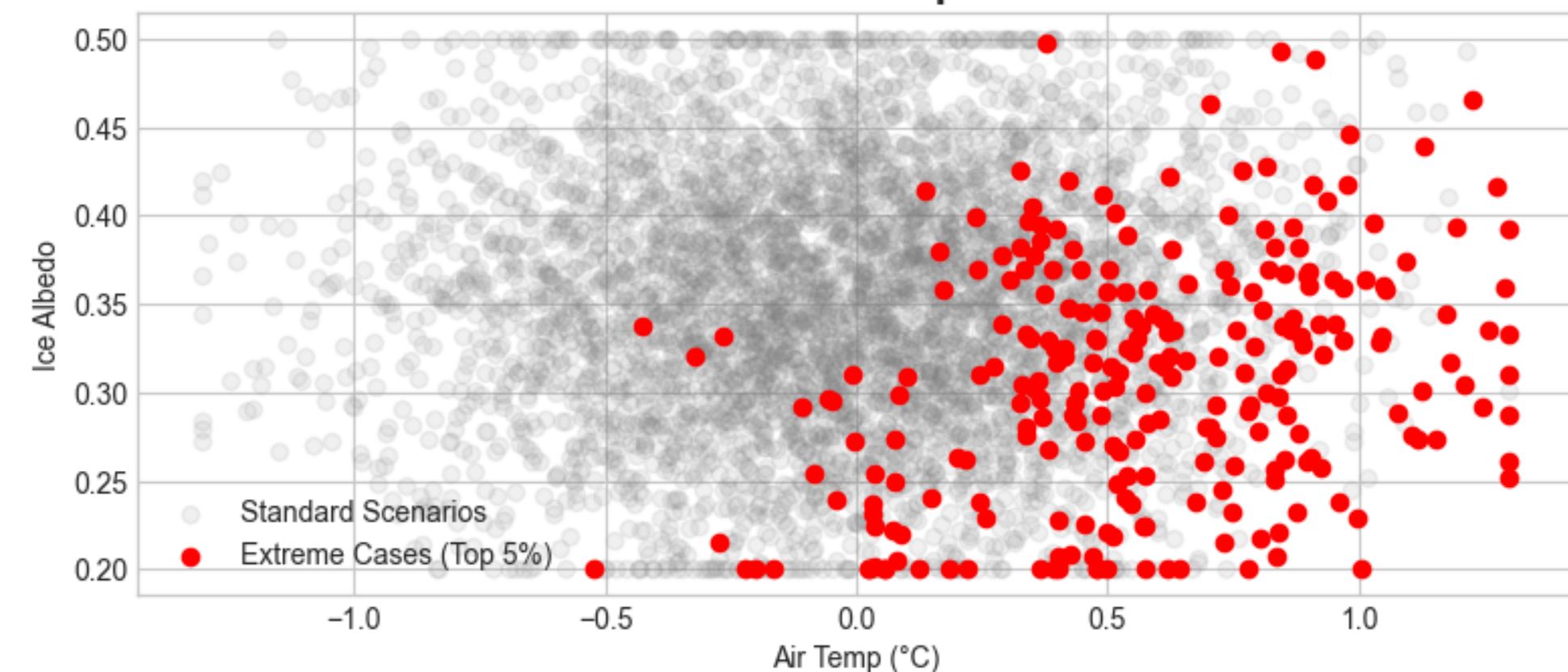
Efficiency: Mathematical Convergence Rates

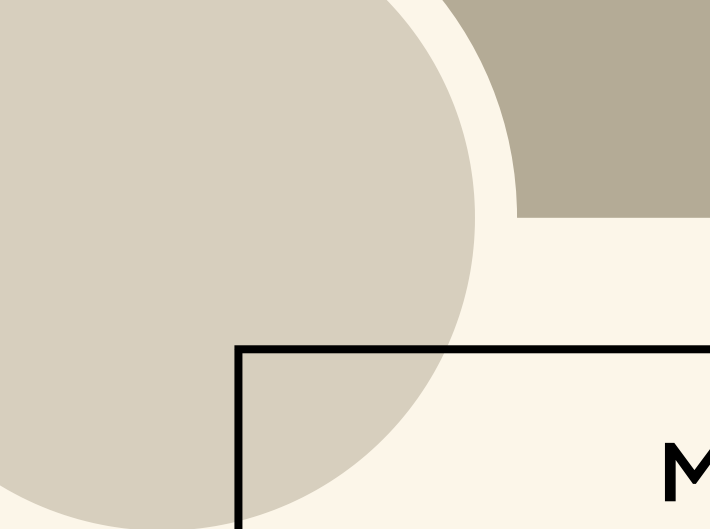


Outcome: Probabilistic Melt Magnitude



Risk Profile: Extreme Temperature & Albedo





Results

Metric	Metric Monte Carlo (M3)	LHS Engine (M4)
Expected Mean Melt	0.02114	0.02123
95 th % Risk Score	0.02817m	0.02892m
Sampling Variance	4.84 x e08	4.67 x e07
Convergence Rate	$O(1/\sqrt{N})$	$\sim O(1/N)$
Sampling Stability	Stochastic Noise	High Stability



Inference- Domain Perspective

- Albedo as a Primary Force Multiplier
- Sensitivity to "Perfect Storm" Scenarios
- The Snowfall Threshold Impact
- Quantifying 95th Percentile Risk



Inference- Computer Science Perspective

- MC: $O(1/\sqrt{N})$ → slow convergence, high variance
- LHS: $O(1/N)$ → faster convergence, low variance
- Stratified sampling → no clustering
- Efficient exploration of 5D input space
- Computational Efficiency through Vectorization
- Reduction in Estimator Variance



References

- Hock, R. (2008). Brief Communication: On the glacier-wide mass balance. *The Cryosphere*, 2(2), 191-200. [Used for: Energy Balance Equations and Melt Parameters].
- Wikipedia (2024). Chebyshev's inequality. Wikipedia, The Free Encyclopedia. [Used for: Statistical bounding and outlier control].
- SciPy Community. (2024). `scipy.stats.qmc.LatinHypercube`. SciPy Documentation. [Used for: Computational implementation of QMC].
- Lumina Decision Systems. (2024). Latin Hypercube vs. Monte Carlo Sampling. Analytica Blog. [Used for: understanding LHS].



Thank You

